

Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерного проектирования
Кафедра проектирования информационно-компьютерных систем

ОТЧЕТ
по лабораторной работе №1
на тему:
**«ПРОЕКТИРОВАНИЕ СЕТЕВЫХ СТРУКТУР
КЛАССОВ С НИЗКОЙ СВЯЗНОСТЬЮ НА JAVA»**

Выполнил

(подпись)

Янович Назар
Олегович
414302

Проверил

(подпись)

Казючиц Владислав Олегович

Минск 2026

ВВЕДЕНИЕ

Создание гибких, масштабируемых и удобных в сопровождении программных систем является одной из важных задач современной разработки программного обеспечения. Для достижения этих характеристик необходимо правильно организовывать взаимодействие между компонентами программы и минимизировать излишние зависимости между ними. Использование слабых связей позволяет снизить влияние изменений отдельных частей системы на остальные компоненты и уменьшить риски.

Слабая связность классов представляет собой подход к построению архитектуры, при котором модули и компоненты имеют минимальное количество прямых зависимостей друг от друга. Взаимодействие между ними осуществляется через простые и стабильные интерфейсы, абстракции и другие механизмы объектно-ориентированного программирования. Такой подход повышает гибкость программной системы и упрощает её изменение.

Цель лабораторной работы – разработка и анализ модели классов с низкой степенью связности на языке Java для изучения принципов построения слабосвязанных архитектур в объектно-ориентированном программировании.

Задачи работы:

- 1 Изучение основных принципов слабой связности в объектно-ориентированном программировании.
- 2 Проектирование структуры классов с минимальным количеством прямых зависимостей.
- 3 Реализация разработанной модели на языке Java.
- 4 Анализ и оценка преимуществ использования слабосвязанной архитектуры.

Преимущества слабой связности:

- 1 Поддерживаемость: изменения в одном модуле оказывают минимальное влияние на другие компоненты системы.
- 2 Гибкость и масштабируемость: добавление новых функций или изменение существующих компонентов может выполняться без существенного изменения всей системы.
- 3 Улучшенная тестируемость: отдельные компоненты могут тестироваться изолированно от остальных частей программы.

Недостатки слабой связности:

- 1 Усложнение архитектуры. Построение системы с низкой степенью связности может требовать использования дополнительных интерфейсов, классов и абстракций.
- 2 Увеличение объёма кода. Для уменьшения прямых зависимостей между компонентами может потребоваться создание дополнительных уровней абстракции и вспомогательных классов.

1 ПРОЕКТИРОВАНИЕ МОДЕЛИ

Main.java

Это точка входа в программу. В данном классе происходит демонстрация работы разработанной модели: создается персонаж, способности, зачарование и различные игровые предметы. Далее способности передаются персонажу для использования, а предметы добавляются в его инвентарь. Класс Main взаимодействует с объектами предметной области, но не содержит сложной логики их работы, так как основная функциональность распределена между соответствующими классами.

Character.java

Этот класс представляет игрового персонажа. Персонаж имеет имя и собственный объект Inventory, который создается непосредственно внутри класса Character. Это демонстрирует композицию, так как инвентарь является составной частью персонажа в рамках разработанной модели.

Также класс Character зависит от интерфейса Ability через метод useAbility. Метод принимает любой объект, реализующий интерфейс Ability, а не конкретную способность. Благодаря этому можно добавлять новые способности без изменения класса Character, что уменьшает связанность между компонентами системы.

Ability.java

Это интерфейс, определяющий общий контракт для всех игровых способностей. Он содержит метод use, который принимает объект Character. Любой класс, реализующий данный интерфейс, обязан предоставить собственную реализацию этого метода.

Использование интерфейса позволяет классу Character взаимодействовать с абстракцией Ability вместо конкретных классов способностей. Такой подход демонстрирует полиморфизм и слабую связность, поскольку новые способности могут быть добавлены в программу без изменения существующей логики персонажа.

Fireball.java, Heal.java и Dash.java

Эти классы являются конкретными реализациями интерфейса Ability. Каждый класс переопределяет метод use и определяет собственное поведение способности.

Fireball представляет способность атаки, Heal - способность восстановления здоровья, а Dash - способность быстрого перемещения. Несмотря на различную реализацию, все способности могут использоваться через общий интерфейс Ability. Это позволяет применять полиморфизм и работать с различными объектами способностей единым способом.

Item.java

Это абстрактный базовый класс для игровых предметов. Он содержит общее поле name и метод getName для получения названия предмета. Также класс содержит абстрактный метод use, который должен быть реализован каждым конкретным типом предмета.

Использование абстрактного класса позволяет выделить общие характеристики всех предметов и одновременно предоставить возможность наследникам реализовать собственное поведение. Благодаря этому классы различных предметов объединяются общей иерархией.

`Weapon.java`, `Armor.java` и `Potion.java`

Эти классы являются наследниками абстрактного класса `Item`. Каждый из них содержит собственные уникальные характеристики и реализует метод `use` по-разному.

`Weapon` содержит значение урона, `Armor` содержит значение защиты, а `Potion` содержит количество восстанавливаемого здоровья. Все объекты могут храниться и использоваться через общий тип `Item`, что демонстрирует наследование и полиморфизм.

`Inventory.java`

Этот класс представляет инвентарь игрового персонажа. Внутри него используется коллекция `List`, предназначенная для хранения игровых предметов различных типов.

Инвентарь содержит ссылки на объекты `Item`, однако не определяет конкретный тип предмета. В коллекции могут одновременно находиться объекты `Weapon`, `Armor` и `Potion`. Это позволяет использовать полиморфизм при работе с предметами.

Связь между `Inventory` и `Item` можно рассматривать как агрегацию. Предметы создаются отдельно от инвентаря и затем передаются в него. Инвентарь не управляет созданием предметов, поэтому они могут существовать независимо от конкретного объекта `Inventory`.

`AppliedEnchantment.java`

Этот класс представляет зачарование, примененное к оружию. Он содержит тип зачарования и метод `activate`, который демонстрирует его активацию.

В разработанной модели объект `AppliedEnchantment` связан с объектом `Weapon`. Оружие содержит примененное к нему зачарование и использует его при выполнении собственного метода `use`. Данная связь демонстрирует композицию, поскольку зачарование рассматривается как составная часть конкретного оружия в рамках разработанной модели.

`Weapon.java` и `AppliedEnchantment.java`

Класс `Weapon` содержит объект `AppliedEnchantment`. При использовании оружия сначала выполняется основная логика оружия, после чего активируется связанное с ним зачарование.

Данная связь позволяет разделить ответственность между классами. `Weapon` отвечает за характеристики и использование оружия, а `AppliedEnchantment` отвечает только за поведение зачарования. Такое разделение делает структуру программы более понятной и упрощает дальнейшее расширение функциональности.

Таким образом, разработанная модель демонстрирует использование основных принципов объектно-ориентированного программирования и различных типов связей между классами. В программе используются

интерфейсы, абстрактные классы, наследование, полиморфизм, агрегация, композиция и зависимости.

UML-диаграмма классов разработанной модели представлена на рисунке 1.1.

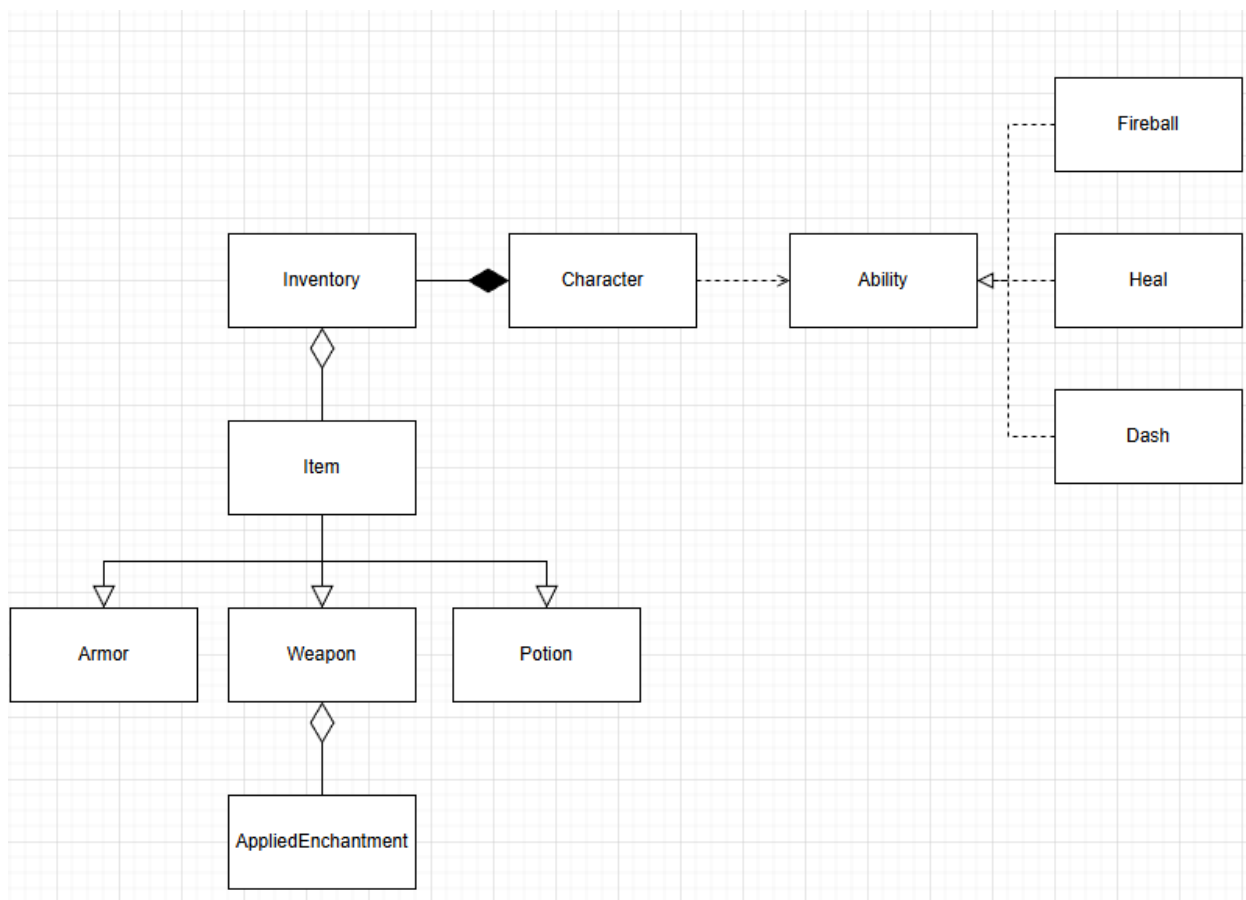


Рисунок 1.1 – UML-диаграмма классов

2 РЕАЛИЗАЦИЯ

Особенности реализации слабых связей в Java представлены ниже.

1 Зависимость (Dependency)

Самая слабая связь. Класс использует другой класс или интерфейс для выполнения определенной задачи, но не хранит объект в качестве постоянного поля. Такая связь обычно возникает через параметры методов, локальные переменные или возвращаемые значения.

В классе `Character` метод `useAbility(Ability ability)` принимает объект типа `Ability` в качестве параметра. Это означает, что `Character` зависит от функциональности способности, однако не зависит от конкретных классов `Fireball`, `Heal` или `Dash`.

Благодаря этому класс `Character` может использовать любую способность, реализующую интерфейс `Ability`. При добавлении новой способности изменять код класса `Character` не требуется.

2 Реализация/Имплементация (Realization/Implementation)

Данная связь возникает между интерфейсом и классом, который реализует этот интерфейс. Класс обязуется предоставить реализацию методов, объявленных в интерфейсе.

Классы `Fireball`, `Heal` и `Dash` реализуют интерфейс `Ability`. Каждый из них предоставляет собственную реализацию метода `use(Character character)`.

Благодаря использованию интерфейса `Ability` различные способности могут использоваться одинаковым образом через общий тип. Это позволяет применять полиморфизм и уменьшает зависимость класса `Character` от конкретных реализаций способностей.

3 Композиция (Composition)

Композиция представляет собой сильную форму отношения "часть-целое". Объект-часть тесно связан с объектом-целым, а его существование в рамках модели определяется существованием основного объекта.

В разработанной модели композиция используется между классами `Character` и `Inventory`. Объект `Inventory` создается как составная часть персонажа и принадлежит конкретному объекту `Character`.

Инвентарь используется для хранения предметов персонажа и является частью его внутренней структуры. Таким образом, связь `Character` и `Inventory` демонстрирует отношение "целое-часть", при котором инвентарь рассматривается как составной компонент персонажа.

4 Агрегация (Aggregation)

Агрегация является более слабой формой отношения "часть-целое". Объекты-части могут существовать независимо от объекта, который их содержит. Объект-целое хранит ссылку на другой объект, но не обязательно управляет его созданием и жизненным циклом.

Класс `Inventory` содержит коллекцию `List<Item>`. Игровые предметы создаются отдельно, например в классе `Main`, после чего передаются в

инвентарь с помощью соответствующих методов. Объекты `Weapon`, `Armor` и `Potion` могут существовать независимо от конкретного объекта `Inventory`.

Также агрегация используется между классами `Weapon` и `AppliedEnchantment`. Объект `AppliedEnchantment` создается отдельно и передается объекту `Weapon`. Оружие использует зачарование при выполнении своего поведения, однако само зачарование не создается внутри класса `Weapon`.

Таким образом, `Weapon` использует объект `AppliedEnchantment`, но не управляет его жизненным циклом. Это позволяет рассматривать данную связь как агрегацию.

5 Наследование (Generalization)

Самая сильная и жесткая форма связи («is a»). Класс-потомок (подкласс) наследует структуру и поведение класса-предка (суперкласса), расширяя или переопределяя его. Это отношение строится на уровне типов, а не экземпляров: потомок является разновидностью родителя. Связь устанавливается на этапе компиляции и не может быть изменена динамически.

В нашем случае данный вид связи используется между абстрактным классом-родителем `Item` и его дочерними классами `Potion`, `Armor` и `Weapon`. От дочерних классов требуется реализация единственного абстрактного метода `use()`, а также они наследуют от родительского класса поле `String name` и связанный с ним метод `String getName()`.

Таким образом, разработанная модель демонстрирует различные виды связей между классами. Использование интерфейсов, абстрактных классов и различных типов отношений позволяет уменьшить количество прямых зависимостей между компонентами и сделать архитектуру программы более гибкой и расширяемой.

ЗАКЛЮЧЕНИЕ

Проектная модель игрового персонажа успешно реализована. Демонстрация принципов слабой связности на языке Java выполнена. Создание модульных, легко заменяемых компонентов подтверждено. Использование интерфейсов, а также применение агрегации и композиции, эффективно снизило зависимость между классами. Полученная структура обеспечивает гибкость и облегчает дальнейшую модификацию кода.

Разработанная модель имеет высокую практическую ценность. Она служит наглядным примером применения принципов объектно-ориентированного проектирования в реальной задаче. Архитектура с низкой степенью связности демонстрирует устойчивость к изменениям и высокую адаптируемость. Навыки, приобретённые при проектировании и реализации, применимы при создании сложных, масштабируемых систем для коммерческого использования.

Дальнейшее развитие модели возможно в нескольких направлениях. Реализация дополнительных видов способностей не составит труда в случае нашей гибкой системы. Интеграция с базой данных позволит реализовать постоянное хранение данных о инвентаре каждого из персонажей и иметь возможность постоянного их редактирования. Модель также может служить основой для изучения работы с системами контроля версий, такими как Git, для демонстрации коллективной разработки.